Campus Café — Serverless Order Tracking on Azure

Project proposal v2 · with disaster recovery and security · Group 6 · George Brown College · Work Integrated Project · October 2026

Serverless | Event-driven (queue trigger) | DR: pilot light in Canada East | Security: 6 layers | est. under $2 / month

Overview

A campus café needs to take customer orders and keep customers informed. This project builds a small order-tracking system on Azure using serverless services. A customer places an order. Café staff move it through four statuses: **Accepted** → **Preparing** → **Ready for pickup** → **Collected**. The customer receives an email at each of the first three.

What is new in v2

- **Disaster recovery (page 3):** data is copied to a second region (Canada East). A stopped copy of the app waits there. If Canada Central goes down, we follow a 4-step runbook. Target: back online within 4 hours.
- **Security (page 4):** six layers of protection, from who can sign in to what code can be deployed.
- **Order of work (page 5):** every new item is tagged NOW free, NEXT cents, or LATER costs money.

How it works (numbers match the diagram on page 2)

1. The customer sends an order: `POST /api/orders` over HTTPS.
2. `SubmitOrder` validates and rate-limits it, saves it as `Accepted` in the Table, and puts a message on Queue `notify`.
3. Staff send a status update: `PUT /status` (function key today, Entra sign-in next).
4. `UpdateOrderStatus` allows only the next valid status, saves it, and queues a message (none for `Collected`).
5. The queue message triggers `SendNotification`. A failed message retries, then moves to the poison queue and raises an alert.
6. `SendNotification` sends the email through Azure Communication Services and marks that status as notified.
7. The customer receives the email (three per order).

Architecture at a glance

- **Primary region, Canada Central:** resource group `rg-campus-cafe` with a Flex Consumption Function App (3 functions), one geo-redundant storage account (Table, Queue, Blob) and Azure Communication Services Email.
- **Platform:** managed identity, Application Insights, Log Analytics, alerts, budget. New: Key Vault, Defender for Cloud, Azure Policy with delete locks, Azure outage alerts.
- **DR region, Canada East:** `rg-campus-cafe-dr` with a stopped Function App, a DR Key Vault and a backup storage account. Built from the same Bicep code.

Design principles

- **Serverless, event-driven:** pay per execution; the queue decouples HTTP from email.
- **Reliable:** retries, poison queue, geo-redundant data, a tested failover runbook.
- **Secure by layers:** identity, API checks, secrets, network, detection, safe deployments.
- **Observable:** Application Insights + Log Analytics; alerts email the team.
- **Cost-aware:** free tiers first; anything that costs real money waits for "later".

Estimated cost per month

Item	Estimate
Azure Functions (Flex Consumption)	Within free grant (250,000 executions, 100,000 GB-s)
Storage (GZRS) + backup storage	Cents (data is a few MB)
Communication Services Email	$0.00025 per email
Application Insights + Log Analytics	Within 5 GB free
Key Vault (primary + DR)	Cents (billed per operation)
DR Function App (stopped)	About $0 while idle
Defender free tier, Policy, alerts, budget	Free; optional log alert about $0.50+
Total	**Under $2, covered by Azure for Students ($100 credit)**

Estimates; confirm in the Azure Pricing Calculator.

Architecture

Primary region with security additions. Tags show order of work: NOW NEXT

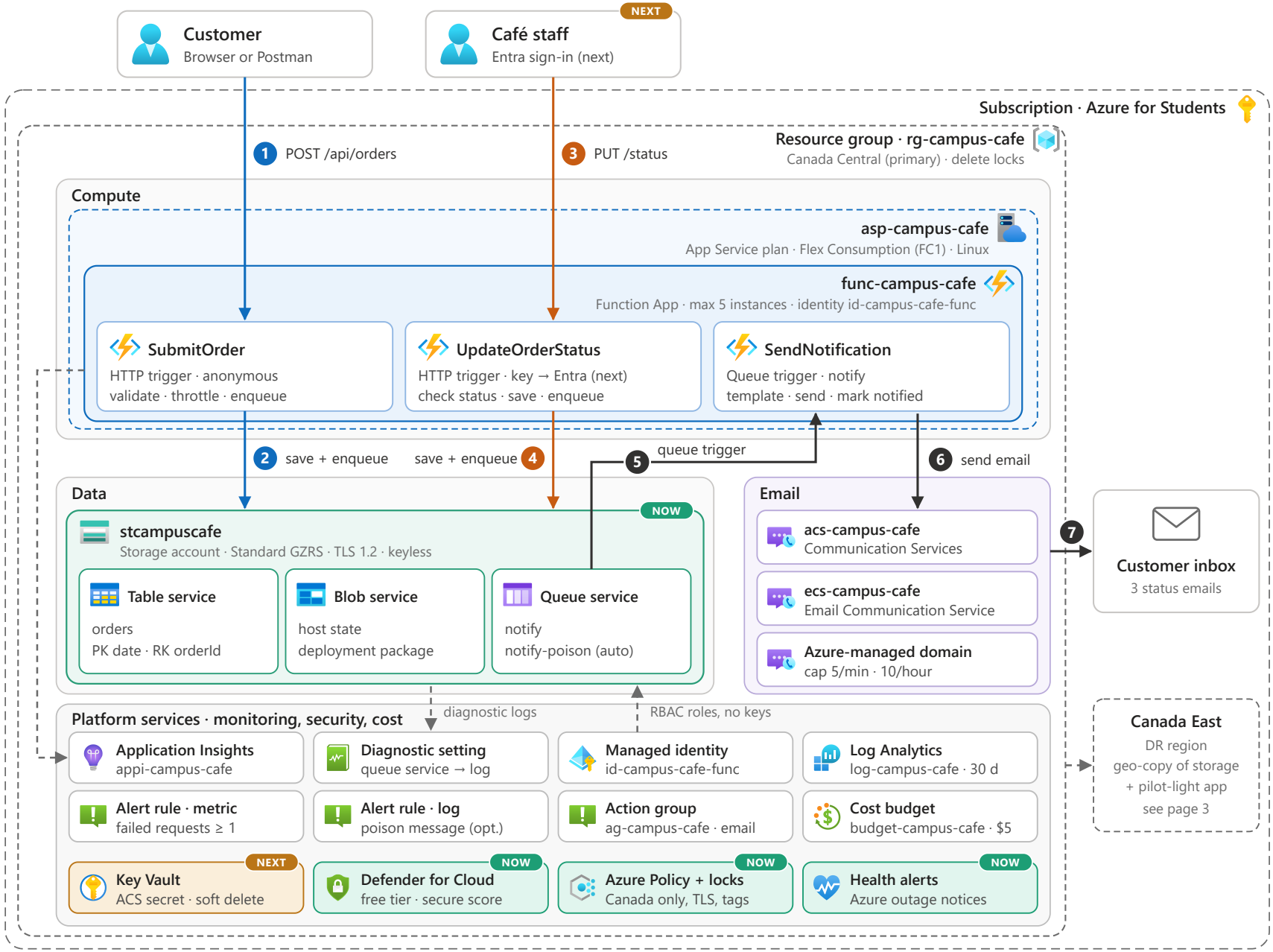


Figure 1. Numbers 1–7 follow the flow on page 1: customer order = 1–2 (blue), staff status update = 3–4 (orange), notification = 5–7 (shared). Dashed lines = identity (RBAC), diagnostic logs and telemetry. The dashed box on the right points to the DR region on page 3.

Disaster recovery strategy

Pattern: backup and restore with a "pilot light" — a cheap, switched-off copy of the app in a second region

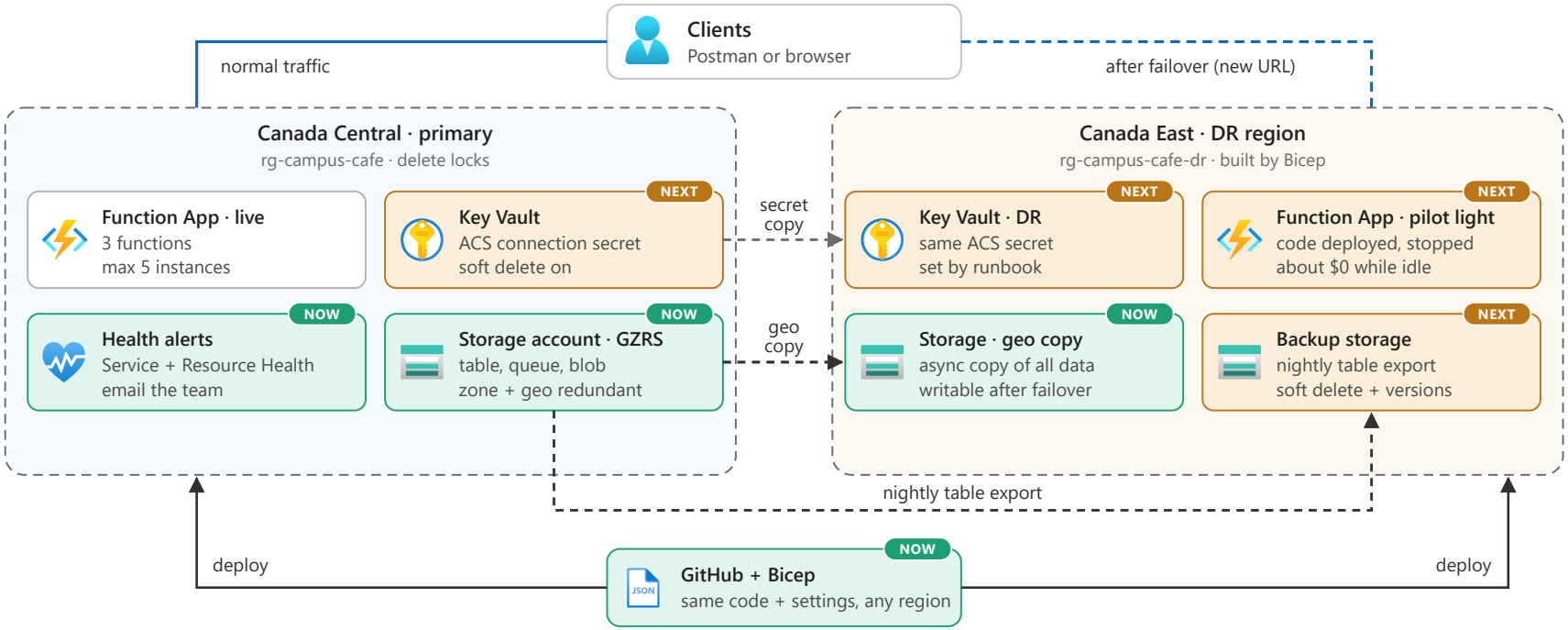


Figure 2. Solid blue = normal traffic. Dashed = data copies and the after-failover path. Tags show order of work.

RTO ≤ 4 hours
How fast we are back online after a region outage.

RPO: minutes
How much recent data we could lose. Geo-copy is not instant and has no time guarantee for tables and queues.

Failover runbook (if Canada Central fails)

1. **Detect:** outage alert arrives. Check the storage account's *Last Sync Time* to see how much data might be missing.
2. **Fail over storage:** start the storage account failover. It usually takes under an hour.
3. **Start the DR app:** switch on the pilot-light Function App (or deploy it with Bicep).
4. **Switch and fix:** give clients the DR URL; re-queue any order whose email was not marked as sent.

After recovery: re-enable geo-redundancy (the account becomes locally redundant after failover), then move back.

What can go wrong, and what protects us

Problem	Protection
A request or instance fails	Automatic retries, then poison queue + alert
One datacenter (zone) goes down	Storage keeps copies in 3 zones (GZRS)
The whole region goes down	Geo-copy in Canada East + pilot-light app + runbook
Someone deletes or breaks data	Delete locks, soft delete, nightly table export
A bad code release	Redeploy the previous version from GitHub with Bicep
The email service is down	Messages wait in the queue and retry later

How we test it

- Talk through the runbook as a team (tabletop).
- Deploy the full stack to Canada East from Bicep, smoke-test, then delete it.
- Practise a *planned* failover on a test storage account: no data loss, geo-redundancy kept.

Security strategy

Defence in depth: six layers, so one mistake does not expose the whole system

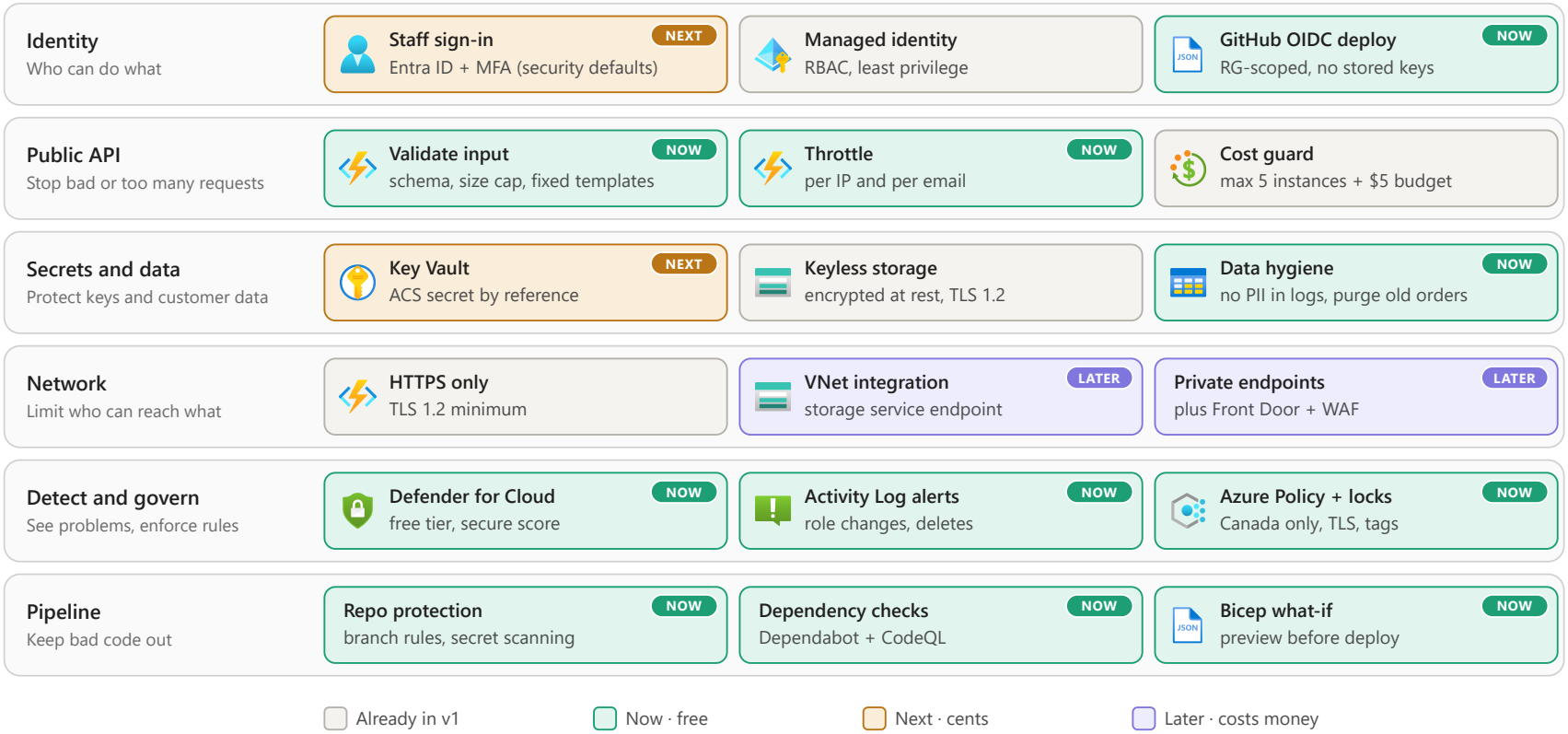


Figure 3. Each box is one control. Grey = already in v1; colour and tag = when we add it.

Threats we cover

Threat	Example in our app	Control
Spoofing	Someone pretends to be staff	Entra sign-in + MFA
Tampering	Order status changed without permission	RBAC, staff-only endpoint, logs
Repudiation	"I never changed that order"	Signed-in identity in logs
Information disclosure	Customer emails leak	TLS, keyless storage, no PII in logs
Denial of service	Flood of fake orders	Throttling, max 5 instances, $5 budget
Elevation of privilege	App gets more access than needed	Least-privilege roles, no stored keys

Two things to know

- **Email abuse:** customers type their own email address, so an attacker could make the café email strangers. Throttling per IP and per email, plus fixed email templates, stop this.
- **Network rules apply to the whole Function App.** We cannot IP-restrict only the staff endpoint without splitting it into its own app, so staff protection comes from sign-in instead.

Free tools first: Entra security defaults (MFA), Defender for Cloud free tier, Azure Policy, GitHub secret scanning.

Order of work and plan

What we build first, what follows, and what waits until there is a budget

NOW about 1 day · free

Disaster recovery

- GZRS storage
- Delete locks + soft delete
- Azure outage alerts
- Bicep + GitHub Actions
- Written runbook

Security

- Input validation + throttling
- No personal data in logs
- Defender for Cloud (free)
- Azure Policy
- GitHub OIDC + secret scanning

NEXT cents

Disaster recovery

- Pilot-light app in Canada East
- DR Key Vault
- Nightly table export + backup storage
- DR drill (planned failover)

Security

- Key Vault for the ACS secret
- Entra sign-in + MFA for staff

LATER costs money

Disaster recovery

- Automatic failover routing (Traffic Manager or Front Door)
- Zone-redundant compute
- Active-active in two regions

Security

- VNet integration + storage service endpoint
- Private endpoints
- Front Door + web application firewall

Build plan

Phase	Work	Output
1 Secure base	Resource group, GZRS storage, monitoring, identity, locks, Policy	Safe, wired environment
2 Core flow	Order + status functions, queue, table, validation, throttling	Statuses advance
3 Email	Notification function, Communication Services, Key Vault	3 customer emails
4 Watch	Alerts, Defender, outage alerts, Activity Log alerts	Problems reach the team
5 Recover	Bicep for Canada East, pilot light, table export, runbook	DR ready
6 Prove it	Demo script, DR drill, README	Live demo and report

Risks and things to confirm

- **Email limits:** the Azure-managed domain allows 5 emails/min and 10/hour. ACS Email retires 30 Sep 2028, so email stays behind one function and can be swapped.
- **Data loss on failover:** the most recent writes may not have reached Canada East yet.
- **To confirm in the portal:** Flex Consumption in Canada East, GZRS in Canada Central, Entra sign-in on Flex, budgets on the student subscription.

References and credits

- **Icons:** Microsoft Azure Public Service Icons V24, file `Azure_Public_Service_Icons_V24.zip` (© Microsoft). Used unmodified, with the product name beside each icon, for architecture documentation, as the pack's Terms of Use permit.
- **Microsoft documentation:** Storage DR and failover · Storage redundancy · Region pairs · Functions reliability · Key Vault reliability · Functions networking · Functions connections and roles · ACS limits · ACS retirement